S10t-D

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

6

Course Code:	CSA1407	**Course Title:**	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	B. Shivakalyan	**Reg. No.:**	192372159
Section / Batch:	2023	**Date:**	16/07/2026

Code of Conduct (192372159)

I B. Shivakalyan *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: BSK

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 /4	3 /4	4 /4	4 /4	3 /4	91.25
2	4 /4	3 /4	3 /4	4 /4	4 /4	88.75
3	3 /4	4 /4	3 /4	4 /4	3 /4	85
4	4 /4	4 /4	4 /4	3 /4	3 /4	91.25
5	3 /4	3 /4	4 /4	4 /4	4 /4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	89.25
	/20	/20	/25	/20	/15	/100

N.B.

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^* abb$ c)$(a^*b^*)^* aba\ (a^* + b^*)^*$ d)$(a+ab)^*$ e) $(aba)^* + (a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d = “xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$:** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.

Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:

1. Keywords: if, else, while, return, int, float
2. Operators: +, -, *, /, =, ==, <, >
3. Separators: (,), {, }, ;, ,
4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123
5. Integer Literals: A sequence of digits, e.g., 12345
6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45

The lexical analyzer needs to tokenize the following input string: int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; }

Questions:

a) How many tokens are there in the given input string?

b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals).

c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

B. Shivakalyan
19737215g
CSA1407.

1) Trance the processing of the expression

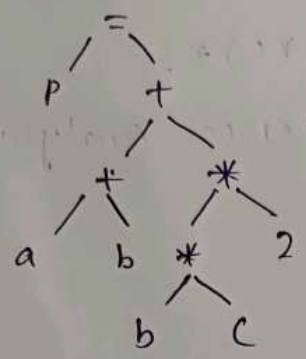
Expression :- p = (a * b) + (b * c) * 2.

a) Tokens generated during Lexical Analysis.

Token	TYpe
p	identifier
=	Assignment operator
(	Left paranthesis
a	identifier
*	Multiplication operator
b	identifier
)	Right parenthesis
+	Addition operator
(	Left parenthesis
b	identifier
*	Multiplication operator
c	identifier
)	Right parenthis
*	Multiplication operator
2	Integer Constant

Total Tokens = 15

b) Syntax Tree:-



c) Intermediate code.

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$

d) Optimized Intermediate code

$t_1 = a * b$

$t_2 = b * c$

$p_1 = t_1 + (t_2 * 2)$

e) Significance of final Target code

* Machine code executable by the processor
* Faster execution after optimization
* Efficient memory & register usage.
* Produces the final executable program

$(aa^*)^*$

$L = \{a^n \mid n \geq 0\}$

Any 5 valid strings

- ε
- a
- aa
- aaa
- aaaa

b) $(b^*)^* abb$

Any Combination of a's & b's followed by abb

$L = \{wabb \mid w \in \{a,b\}^*\}$

Ay Five valid strings

- abb
- aabb
- babb
- aaabb
- bbabb

c) $(a^*b^*)^* aba (a^*+b^*)^*$

All strings over $\{a,b\}$ Containing the sub string "aba"

Any 5 valid strings

- aba
- aaba
- abab

- abaa
- babab

d) $(ab)^*$

Strings formed by repeating either a or ab any number of times

Any 5 valid strings

- ε
- a
- ab
- aa
- aab

e) $(aba)^* + (a^*b^*)^*$

- Repeated occurrences of aba, or
- Any string over $\{a,b\}$ (since $(a^n)^{n^*} = (a+b)^{n^*}$).

Hence

$L = \{a,b\}^*$

Any 5 valid strings

- ε
- a
- b
- ab
- ba

inal Answer

Question	Language
(a) $(\varepsilon + aa^*)^*$	All strings of only a's
(b) $(a^n)^n abb$	Any strings ending with abb
(c) $(a^n)^n + b^n *)^n *$	All strings containg ab
(d) $(a + ab)^+$	Strings Formed by repeating a or ab
(e) $(aba)^n * + (a^n)^n +$	All strings over $\{a, b\}$

3) i) Find the Number of Tokens in the given C Statement:

```
main()
{
    int a = 10;
    char b = "abc";
    int c = 30;
    char d = "xyz";
    int m = 40.5; }
```

NO	Token
1	main
2	(
3	)

4	{
5	int
6	a
7	=
8	10
9	;
10	char
11	b
12	=
13	"abc"
14	;
15	int
16	c
17	=
18	30
19	;
20	Char
21	d
22	=
23	"XYZ"
24	;
25	int
26	m
27	=
28	40.5
29	;
30	}

Total No of Tokens = 30

Identify the Lexical Errors

```
void main()
{
   int X = 10, Y = 20;
    Char * a;
    a = & X;
    X = 1xab;
}
```

Lexical Error

Statement	Lexical Error	Reason
X = 1xab;	1xab	invalid numeric constant identifier cannot start with a digit and 1xab is neither a valid integer nor a valid hexadecimal

Not Lexical Error :-

- void → valid keyword
- main → valid identifier
- char * a; → valid declaration
- a = & x; → Tokens are lexically valid (although it is a semantic / type error because char * is assigned the address of an int)

Note :- a = & x ; is not a lexical error, it is a semantic / type error.

4) i) Language of all strings having an even number of 0's

Regular Expression :-

$1^*(01^*01^*)^*$

ii) Language of all strings containing at least one 1

Regular Expression :-

$(0+1)^* 1 (0+1)^*$

iii) Language of all strings in which both the number of 0's & the number of 1's are odd.

A standard regular expression 21 :

$(00+11)^*(01+10)(00+11)^*(01+10)(00+11)^*(01+10)(00+11)^*)^*$

This generates exactly the strings where both the numbers of 0's & 1's are odd

iv) Regular Expression :-

$1(0+1)^*0$

v) Language of all strings that do not contain Consecutive 1's :-

$(0+10)^*(\varepsilon+1)$

a) How many Tokens are there in the given input string

```
int var123 = 12345;
float var456 = 123.45;
if (var123 == var456) { return; }
```

Token list

No	Token	Type
1	int	keyword
2	var123	identifier
3	=	operator
4	12345	integer literal
5	;	Separator
6	float	keyword
7	var456	identifier
8	=	operator
9	123.45	floating-point literal
10	;	Separator
11	if	keyboard.
12	(	Separator
13	if	keyword

14	==	operator
15	var456	identifier
16	)	Separator
17	{	Separator
18	return	keyword
19	;	Separator
20	}	Separator

Total No. of Tokens = 20

5) b) Categorize the Tokens:-

1) keywords
- int
- float
- if
- return

2) operators
- =
- =
- ==

3) Separator
- ;
- ;
- (
-)
- {
- ;
- }

4) Identifier
- var123
- var456
- var123
- var456

5) integer literals
- 12345

6) Floating-point literals
- 123.45

FSM State Transitions

Assume:

- Single-character operators/separators require 1 transition
- 2-character operator (==) requires 2 transition

Transition Count

Token	character	Transition
int	3	3
var123	6	6
=	1	1
12345	5	5
;	1	1
float	5	5
varurb	6	6
=	1	1
123.45	6	6
;	1	1
if	2	2
(	1	1
var123	6	6

)	1	1
{	1	1
return	6	6
;	1	1
}	1	1.

BRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation